



Московский
транспорт



Молоток

Кейс 05 · решение ReSense

Обнаружение посторонних объектов в тоннеле метро по данным 3D-лидара

О команде

- **Капитан:** <ФИО>, <специальность>
- **Кол-во участников:** 4 человека
- **Краткое описание:**
 - <как образовалась команда>
 - место учёбы: <место учёбы / работы>
- **Город и регион:** <город>

КОМАНДА «Молоток»

Краткое описание решения:

ROS 2-модуль: по потоку 3D-лидара 10 раз в секунду строит модель «нормального тоннеля» — полотно, рельсы, ось пути с кривизной по стенам, — проверяет габарит поезда 2,1 × 3,0 м и отвечает GO / CAUTION / STOP / FAULT с расстоянием до препятствия. Решает геометрия; обученное «мнение» о треке лишь откладывает сомнительный STOP, не больше чем на ~1 с. Без GPU, в Docker.

Уникальность решения:

Описываем среду, а не объекты: крепление лидара и ось пути калибруются по рельсам в каждом кадре, кривизна — по стенам и рядам колонн, поэтому коридор осмыслен и там, где рельсов уже не видно. Публикуем оценку дальности контроля; возможны пропуски объектов. Настройка и оценка — на препятствиях, вставленных трассировкой лучей в реальные кадры.

КОМАНДА «Молоток»



<Имя
Фамилия>

- Капитан · ROS 2, Docker, интеграция
- <@ник>
- <место учёбы / работы>

<Имя
Фамилия>

- Визуализация, демо, презентация
- <@ник>
- <место учёбы / работы>

<Имя
Фамилия>

- Компьютерное зрение: модель пути, трекинг
- <@ник>
- <место учёбы / работы>

<Имя
Фамилия>

- Данные, синтетика, метрики, тесты
- <@ник>
- <место учёбы / работы>

Краткая история команды:

01

Четыре инженерных направления — ROS 2 и интеграция, визуализация, компьютерное зрение, данные и тесты — собраны вокруг задачи контроля габарита тоннеля по 3D-лидару.

Почему вы выбрали именно эту задачу из предложенных на хакатоне?

02

В записях почти нет реальных препятствий, а станции и стрелки похожи на объекты. Поэтому система оценивает геометрию тоннеля и ищет препятствия в габарите; пропуски возможны и в пределах расчётной дальности контроля.

С какими основными сложностями или вызовами вы столкнулись и как их преодолели?

03

Препятствий в записях почти нет — «ставили» людей и ящики в реальные кадры трассировкой лучей лидара; эти примеры оцениваются отдельно от реальных записей. Станции и стрелки давали ложные остановки — помогли ось по стенам, зона доверия и подтверждение 0,5 с. Записи разные (два топика, 120° и 360°, наклон крепления 3°) — узел сам находит вход и калибруется по рельсам.

Техническая суть решения

- ROS 2 Humble-узел на Python (numpy / scipy / scikit-learn) и необязательные ядра на C++
- Вход — PointCloud2 любого из двух наборов топик / frame_id; выход — решение GO / CAUTION / STOP / FAULT, дистанция до препятствия и оценка дальности контроля, Detection3DArray, маркеры RViz, JSON-статус
- В каждом кадре: автокалибровка крепления → модель пути (полотно, рельсы, ось, кривизна по стенам) → габарит 2,1 × 3,0 м + низкие объекты на рельсах → кластеры → фильтры инфраструктуры → подтверждение 0,5 с и «мнение» о треке
- 18-23 мс на кадр (p95 ≤ 33 мс; с декодированием облака p95 ≤ 48 мс) на одном ядре с ядрами на C++ (они сокращают время детектора на 38-57 %, выход тот же)
- Все 13 759 реальных кадров: человек на пути найден в 61 из 61 кадра (от рельсов — 58), 2,5 ложных события на км поездки — на ней же подбирались правила (на кусках, которых модель не видела, — 2,8)
- Только CPU: GPU оценили и не берём — выигрыш ≤ 30-45 мс на кадр, а контейнер с GPU не стартует без nvidia-container-toolkit

Маркетинговая суть решения

Применение: «взгляд вперёд» для беспилотного поезда (GoA3/4) и хозяйственных поездов; контроль габарита при обкатке линий

Развитие: скорость поезда от одометрии — узел её уже принимает (своя оценка по лидару: ошибка 0,07 м/с, пока опция); опора высоты по своду тоннеля — мелкие объекты дальше 100 м; «мнение» о треке — переобучить на реальных препятствиях

Внедрение: docker load → docker run → ros2 bag play или одной командой scripts/play_bag.sh; один файл параметров; стандартные сообщения ROS 2; адаптер торможения и проверка свежести — отдельный этап интеграции



Проблема

Поезд без машиниста должен видеть путь: 100 м — хорошо, 200 — очень хорошо, 300 — отлично. При 80 км/ч и замедлении 1–1,3 м/с²: 190–250 м без задержек. Тоннель почти всегда пуст: классов объектов для обучения нет.

Альтернативы

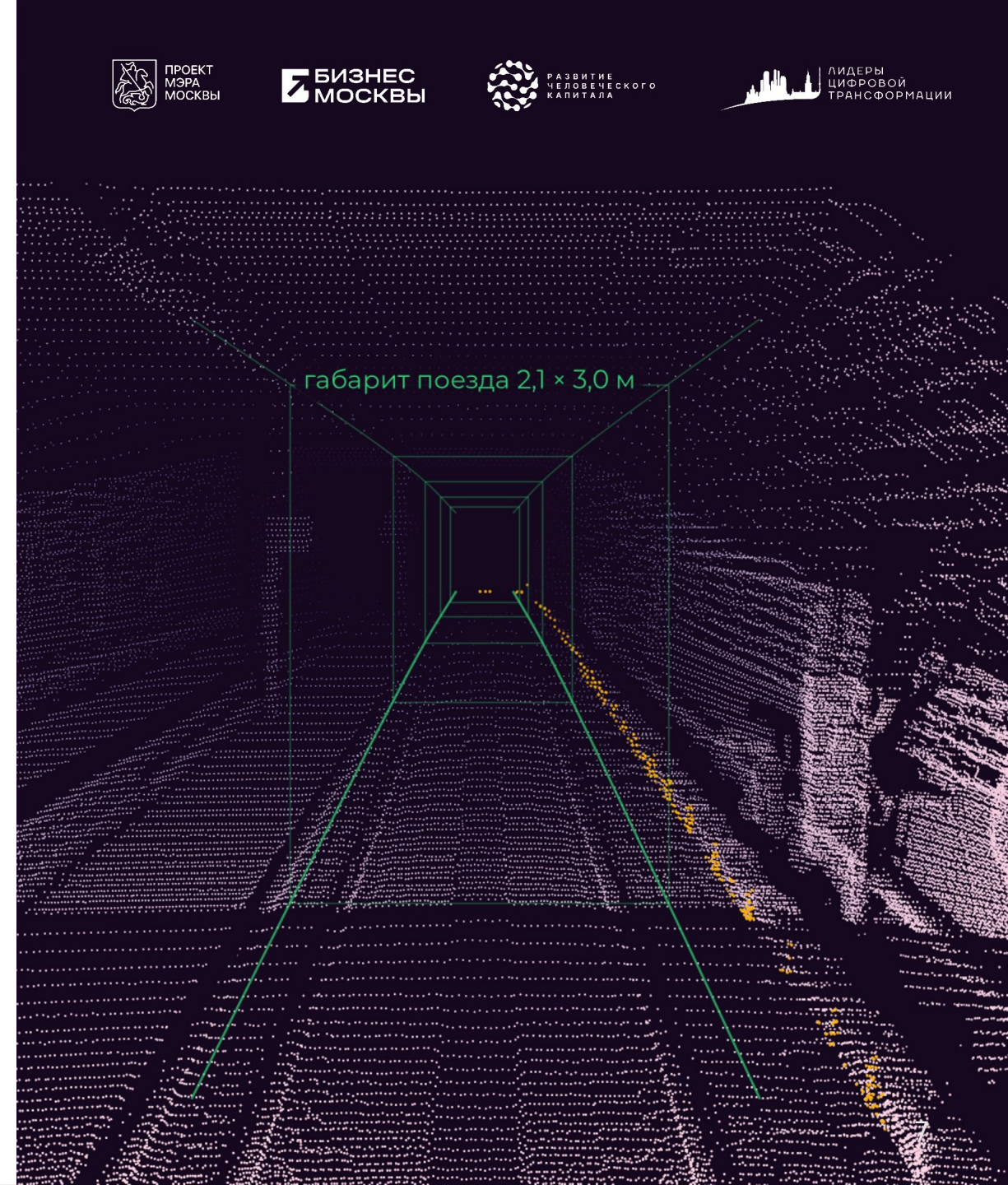
Обученный 3D-детектор требует размеченных препятствий; качество таких моделей на наших тоннелях не измерялось. Сравнение с картой требует локализации и повторного проезда.

Наше решение

Описать нормальный тоннель и искать препятствия в габарите поезда 2,1 × 3,0 м. Ось строится по рельсам, полотну и стенам без карты; обученное «мнение» может отложить сомнительный STOP. Есть пропуски и ложные тревоги.

ДАННЫЕ И СЕНСОР

- **13 759 кадров** — 6 записей (2 488) и 20-минутная поездка (11 271 кадр, 13 км, 7 остановок, кривые до $R \approx 350$ м)
- **Hesai Pandar128**, 10 Гц, ~190 тыс. точек в кадре: стены видны на 150–200 м, полотно — до ~100 м
- **~210 м — предел отражений в тоннеле:** каждая запись обрывается на 209–210 м, дальше нет ни одной точки (паспорт: 200 м при 10 % отражения)
- Объект 0,5 м даёт ~7 точек на 100 м и 1–2 точки на 200 м
- Реальные препятствия: человек и предмет на рельсе в записи doubleT_obstacle; всё остальное — синтетика в реальных кадрах, каждое число помечено



АЛГОРИТМ



Автокалибровка

вертикальная ось вращения;
ориентация и наклон — по
рельсам и полотну

1

Габарит

коридор 2,1 × 3,0 м вдоль оси
и ступень низких объектов на
рельсах

3

Решение

подтверждение 0,5 с и
«мнение» о треке → GO /
CAUTION / STOP / FAULT и
дальность контроля

5

Модель пути

полотно, головки рельсов, ось;
кривизна по стенам и
колоннам до 150–200 м

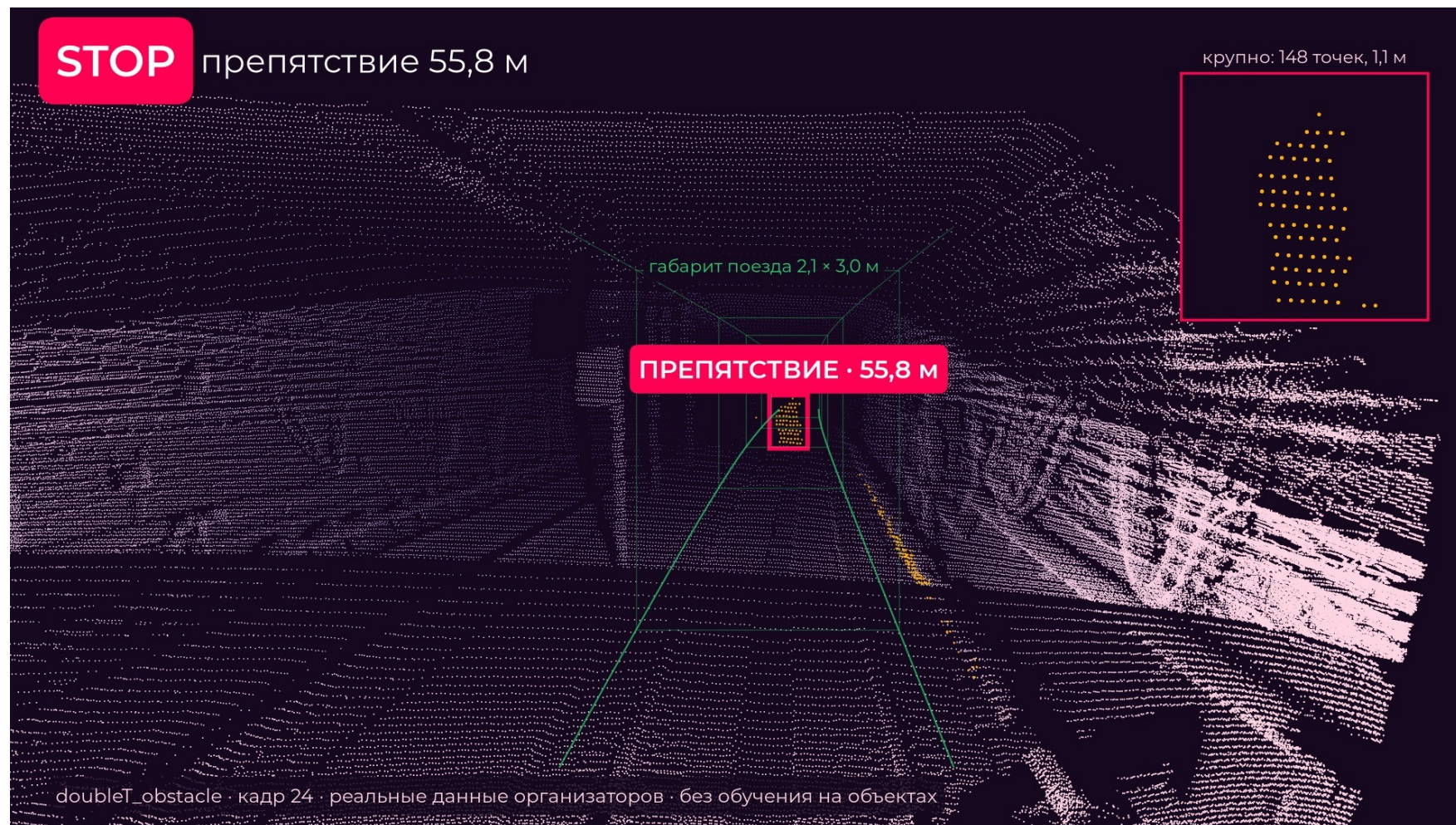
2

Кластеры

радиус растёт с дальностью;
фильтры инфраструктуры;
дальнее правило

4

ГЛАВНЫЙ КАДР



Реальная запись организаторов

Человек переходит путь на 55-57 м

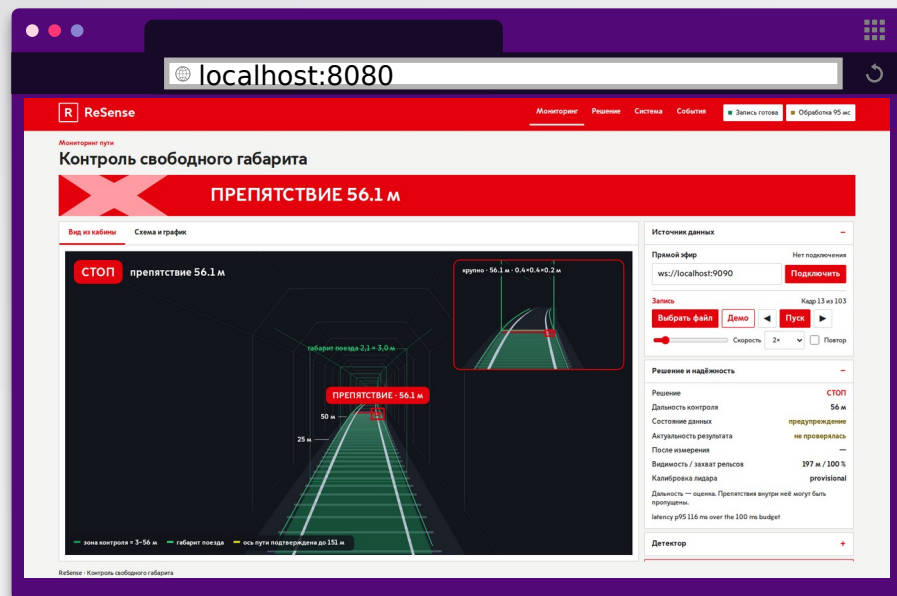
Найден в **61 из 61** кадра внутри габарита (от рельсов — 58)

Тревога с первого кадра в габарите

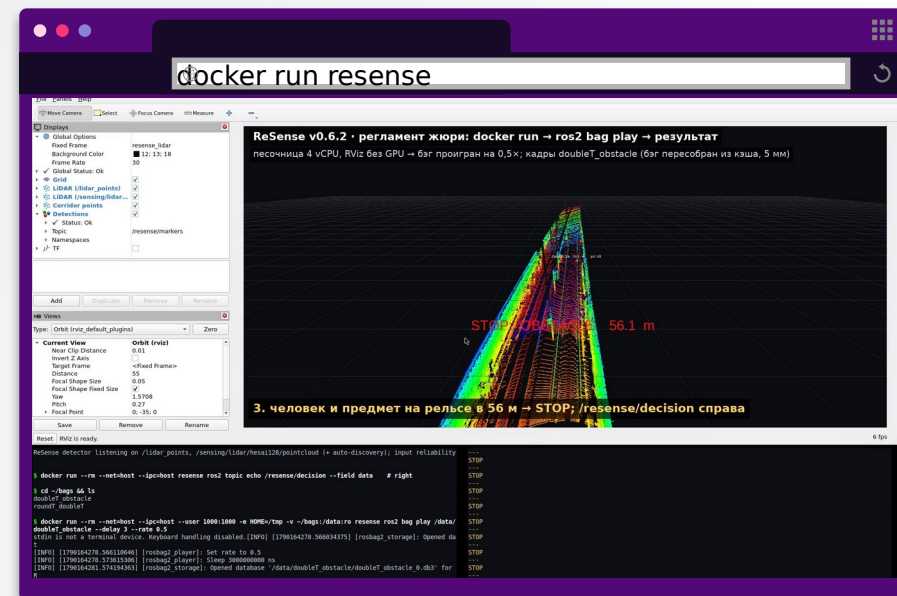
Ошибка дальности $\leq 0,23$ м

Предмет на рельсе — в **125 из 126** кадров после ухода человека

ДЕМОНСТРАЦИЯ



- Веб-дашборд «Контроль свободного габарита»: вид из кабины по статусу узла из Docker-прогона на doubleT_obstacle — STOP 56,1 м, оценка дальности контроля 56 м; живой узел — через rosbridge



- Цепочка жюри в Docker: узел с RViz, bag play от обычного пользователя из другого контейнера, /resense/decision — STOP 56 м (видео 69 с)

docker load → docker run → ros2 bag play → /resense/decision
или одной командой: `scripts/play_bag.sh путь_к_бэгу [--archive resense-image-версия.tar.gz]`

13 759

реальных кадров прогнаны целиком

61 из 61

кадров с человеком на пути — тревога (от рельсов — 58; остальные — по оси лидара, от которой меряют организаторы)

2,5 на км

ложных событий в поездке (32 за 13 км) — на ней же подбирались правила; на кусках поездки, которых модель не видела, — 2,8 на км (измерено)

Реальная запись: человек на пути — 61 из 61 кадра (от рельсов — 58), тревога с первого кадра в габарите, ошибка $\leq 0,23$ м; предмет на рельсе — 125 из 126 кадров

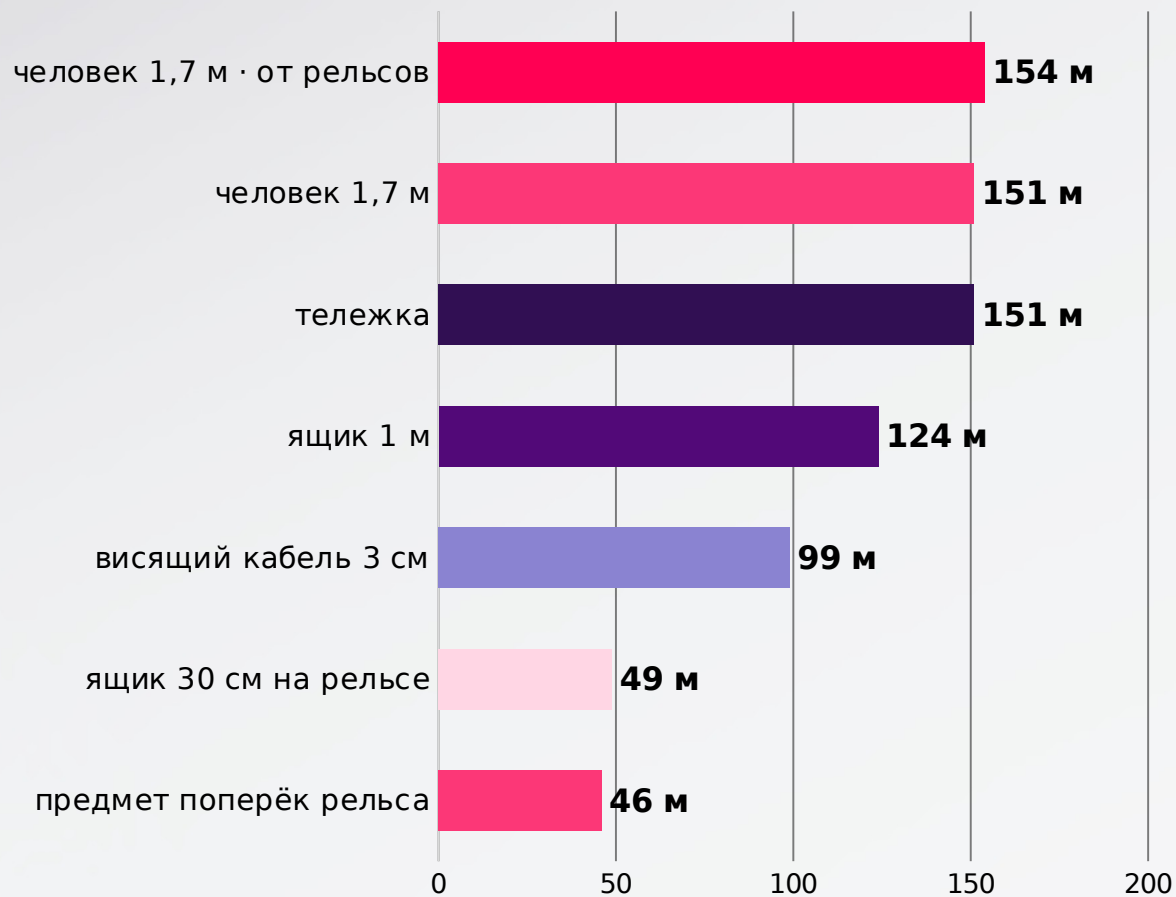
Ложные остановки (реальные, на данных настройки правил): поездка 13 км — 32 события (без «мнения» — 43; на кусках, которых модель не видела, — 37); пять пустых записей — 11

Объекты организаторов [их синтетика]: ящик 2×2 м — с 98 м, доска — с 87 м, у края — с 35 и 18 м; на новых местах [проверка заявлена заранее] — 723 из 2 458 кадров (было 544)

Человек на подходе [наша синтетика]: первое подтверждение 151 м (в 5 парах: $150 \rightarrow 154$ м от ближних рельсов), в каждой полосе 10 м — со 115 м; 167 м со скоростью поезда

Задержка 18–23 мс на кадр ($p95 \leq 33$ мс) на одном ядре с ядрами на C++; с декодированием облака $p95 \leq 48$ мс; через ROS в Docker на 4 ядрах — $p95$ 58–81 мс (замер 26.09); без GPU

ДАЛЬНОСТЬ



первое подтверждение на прямой, медиана 6 подходов (прогон гейта 27.09; ящик 30 см и предмет поперёк рельса — 24.09); «от рельсов» — объект поставлен по ближним рельсам, а не по дальней оси детектора (5 пар, прежняя постановка — 150 м); поезд 17-21 м/с без датчика скорости · синтетика в реальных кадрах поездки

- **~210 м — предел отражений в тоннеле**

дальше 210 м во всех 13 759 кадрах нет ни одной точки

- **Со скоростью поезда**

накопление 5 кадров: человек 167 м; своя скорость по лидару (ошибка 0,07 м/с) STOP раньше не даёт — опция

- **Устойчиво**

человек в $\geq 90\%$ кадров каждой полосы 10 м — со 115 м

- **В кривых**

6 из 7 подходов, с 58-86 м — предел видимости за стеной ($R \approx 350$ м)

ОБЪЕКТЫ ОРГАНИЗАТОРОВ



первый STOP, м; синевым — без устойчивого STOP · cloud_with_fake_obj: 10 объектов, 1 510 кадров, поезд едет к ним 1,4–20 м/с, скорость узлу не дана · синтетика организаторов

• STOP у 8 из 8, у края — с 18–35 м

ящик 2 × 2 м в центре — с 98 м, с первого появления; доска поперёк рельсов — с 87 м; кубы 0,3 м — с 43–56 м. На новых местах (72 случая, проверка заявлена заранее) — 723 из 2 458 кадров (было 544); на том же фоне без объекта — 0

• Мелкое вдали — поздно

на 60–115 м от куба 0,3 м 2–4 точки в кадре: дальше 60 м трек начинаем и с 4, но STOP — только если объект приближается, иначе CAUTION; ящик у верха габарита — STOP в 56 из 124 кадров — непрерывно с 111 м, дальше CAUTION или ничего

• Ограничения — говорим честно

ближе 60 м на прямой точка в габарите и по оси лидара, от которой ставили объекты (–0,24° к рельсам; вопрос организаторам открыт); от рельсов габарит не сужается. GO с дальностью контроля за объектом — 16 кадров (было 46); ложных STOP: 6 у ящика снаружи, 0 вне объектов

01

Калибровка

- 8 проверенных ориентаций вокруг вертикальной оси; крен и тангаж по рельсам и полотну; не любое крепление

02

Свежесть данных

- нет актуального статуса → FAULT или удержание STOP; потребитель проверяет /resense/status своим таймером

03

Любой вход

- оба набора топик / frame_id, поиск топика, перезапуск на новой записи

04

Воспроизводимо

- 731 тестов на путях pipру и C++ (выход совпадает бит в бит); CI собирает образ и проигрывает бэги через узел; цепочка организаторов прогнана в Docker на реальных записях

Полотно полно предметов. «Всё выше полотна» — 1 482 ложных события за 20 мин; порог « ≥ 3 см над головкой рельса + 0,5 с» — единицы; мелкие предметы на полотне между рельсами (кубы 0,3 м; ящик 0,5 м найден в 1 из 6 подходов) — слепое пятно

Предмет на рельсе делился на две ступени. Верх на 0,10–0,15 м — между ступенью низких объектов и габаритом; кластеризуем его целиком: 2 → 125 из 126 кадров

Станции и стрелки. Край платформы и конструкции у торца — главный источник ложных остановок; без рельсов в ближней зоне дальше 40 м — только предупреждение; обученное «мнение» о треке может отложить сомнительный STOP — не больше 10 кадров (~1 с) за жизнь трека, не ближе 25 м и не для стоящего тела ≥ 1 м ближе 40 м

Дальше 100 м — 1-5 точек на объект. Высокие объекты — по дальнему правилу до ~150 м; мелкие (< 0,6 м) — ближе: кубы 0,3 м организаторов — только с 43–56 м (на 60–115 м — 2–4 точки)

Объекты организаторов у края и тонкие. Их объекты стоят от оси лидара ($-0,24^\circ$ к рельсам): ближе 60 м на прямой точку засчитываем и по ней — у края STOP с 35 и 18 м (габарит шире до 0,2 м, нигде не уже); висящий предмет 5 см — STOP только с 30 м

01

Что получилось

- ROS 2-модуль в Docker; все 13 759 реальных кадров: человек и предмет на рельсе найдены, 2,5 ложных события на км (на данных, где подбирались правила; на невиданных кусках поездки — 2,8); ящик организаторов — с 98 м; 731 тестов и CI

02

Что дальше

- мелкие объекты раньше: короткие сигнатуры инфраструктуры, опора высоты по своду тоннеля; скорость от одометрии (своя по лидару — опция); «мнение» о треке — переобучить на реальных препятствиях; ответ организаторов: габарит от рельсов или от оси лидара

03

Внедрение

- `docker load → run → ros2 bag play` или одной командой: `scripts/play_bag.sh`
`путь_к_бэгу [--archive resense-image-версия.tar.gz]` — образ, узел, запись и каждая смена решения с дистанцией; один файл параметров; стандартные сообщения ROS 2; адаптер к торможению ещё нужно разработать и проверить